# PRAKTIKUM 5
# DOUBLY LINKED LIST

**NAMA:** Ihsanta Zaki Sanjaya

**KELAS:** 1 D4 Teknik Komputer B

**NRP:** 3223600058

**DOSEN:** Dr Bima Sena Bayu Dewantara S.ST, M.T

**MATA KULIAH:** Praktikum Pemograman Dasar 2

**TANGGAL:** Rabu, 17 April 2024

**D4 TEKNIK KOMPUTER**
**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**
**2024**

# Praktikum 5
# Doubly Linked List

Untuk memberikan pemahaman kepada anda tentang konsep dasar Doubly Linked List dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

1. **Percobaan 1 – Menambahkan Node Pada Doubly Linked List**
   Pada percobaan ini, sebuah Doubly Linked List (DLL) dideklarasikan dan diinisialisasi.

```
// A complete working C program to demonstrate all insertion methods
#include <stdio.h>
#include <stdlib.h>

// A linked list node
struct Node {
	int data;
	struct Node* next;
	struct Node* prev;
};

/* Given a reference (pointer to pointer) to the head of a list and an int, inserts a new
node on the front of the list. */
void push(struct Node** head_ref, int new_data)
{
	/* 1. allocate node */
	struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

	/* 2. put in the data */
	new_node->data = new_data;

	/* 3. Make next of new node as head and previous as NULL */
	new_node->next = (*head_ref);
	new_node->prev = NULL;

	/* 4. change prev of head node to new node */
	if ((*head_ref) != NULL)
		(*head_ref)->prev = new_node;
```

```
	/* 5. move the head to point to the new node */
	(*head_ref) = new_node;
}

/* Given a node as prev_node, insert a new node after the given node */
void insertAfter(struct Node* prev_node, int new_data)
{
	/*1. check if the given prev_node is NULL */
	if (prev_node == NULL) {
		printf("the given previous node cannot be NULL");
		return;
	}

	/* 2. allocate new node */
	struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

	/* 3. put in the data */
	new_node->data = new_data;

	/* 4. Make next of new node as next of prev_node */
	new_node->next = prev_node->next;

	/* 5. Make the next of prev_node as new_node */
	prev_node->next = new_node;

	/* 6. Make prev_node as previous of new_node */
	new_node->prev = prev_node;

	/* 7. Change previous of new_node's next node */
	if (new_node->next != NULL)
		new_node->next->prev = new_node;
}

/* Given a reference (pointer to pointer) to the head of a DLL and an int, appends a
new node at the end */
void append(struct Node** head_ref, int new_data)
{
	/* 1. allocate node */
	struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

	struct Node* last = *head_ref; /* used in step 5*/
```

```
	/* 2. put in the data */
	new_node->data = new_data;

	/* 3. This new node is going to be the last node, so
		make next of it as NULL*/
	new_node->next = NULL;

	/* 4. If the Linked List is empty, then make the new
		node as head */
	if (*head_ref == NULL) {
		new_node->prev = NULL;
		*head_ref = new_node;
		return;
	}

	/* 5. Else traverse till the last node */
	while (last->next != NULL)
		last = last->next;

	/* 6. Change the next of last node */
	last->next = new_node;

	/* 7. Make last node as previous of new node */
	new_node->prev = last;

	return;
}

// This function prints contents of linked list starting from the given node
void printList(struct Node* node)
{
	struct Node* last;
	printf("\nTraversal in forward direction \n");
	while (node != NULL) {
		printf(" %d ", node->data);
		last = node;
		node = node->next;
	}

	printf("\nTraversal in reverse direction \n");
```

```
	while (last != NULL) {
		printf(" %d ", last->data);
		last = last->prev;
	}
}

/* Drier program to test above functions*/
int main()
{
	/* Start with the empty list */
	struct Node* head = NULL;

	// Insert 6. So linked list becomes 6->NULL
	append(&head, 6);

	// Insert 7 at the beginning. So linked list becomes 7->6->NULL
	push(&head, 7);

	// Insert 1 at the beginning. So linked list becomes 1->7->6->NULL
	push(&head, 1);

	// Insert 4 at the end. So linked list becomes 1->7->6->4->NULL
	append(&head, 4);

	// Insert 8, after 7. So linked list becomes 1->7->8->6->4->NULL
	insertAfter(head->next, 8);

	printf("Created DLL is: ");
	printList(head);

	getchar();
	return 0;
}
```

Hasil percobaan:

```
Praktikum > P5 1.1.cpp > append(Node **, int)
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node*next;
    struct Node* prev;
};
void push(struct Node** head_ref, int new_data) {
    struct Node* new_node=(struct Node*)malloc(sizeof(struct Node));
    new_node->data=new_data;
    new_node->next=(*head_ref);
    new_node->prev=NULL;
    if((*head_ref)!=NULL)
        (*head_ref)->prev=new_node;
    (*head_ref)=new_node;
}

void insertAfter(struct Node* prev_node, int new_data) {
    if(prev_node==NULL){
        printf("The given previous node cannot be NULL");
        return;
    }
    struct Node* new_node=(struct Node*)malloc(sizeof(struct Node));
    new_node->data=new_data;
    new_node->next=prev_node->next;
    prev_node->next=new_node;
    new_node->prev=prev_node;
    if(new_node->next !=NULL)
        new_node->next->prev=new_node;
}
void append(struct Node** head_ref, int new_data) {
    struct Node* new_node=(struct Node*)malloc(sizeof(struct Node));
    struct Node* last=*head_ref;
    new_node->data=new_data;
    new_node->next=NULL;
    if(*head_ref==NULL){
        new_node->prev=NULL;
        *head_ref=new_node;
        return;
    }
    while(last->next !=NULL)
        last=last->next;
        last->next=new_node;
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop
\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapter
s\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-jfssz
ftw.uye' '--stdout=Microsoft-MIEngine-Out-msuu0nir.fm0' '--stderr=Mi
crosoft-MIEngine-Error-ebuesdyg.dtc' '--pid=Microsoft-MIEngine-Pid-o
uh3oq0b.rzx' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=
mi'
Created DLL is:
Traversal in forward direction
 1 7 8 6 4
Traversal in reverse direction
 4 6 8 7 1
```

Output program:

```
Created DLL is:
Traversal in forward direction
 1 7 8 6 4
Traversal in reverse direction
 4 6 8 7 1
```

Dari percobaan di atas, cobalah menambahkan beberapa node baru secara acak pada bagian tengah sebelum node lain, belakang, depan dan tengah setelah node lain! Tambahkan program anda untuk menampilkan atau mencetak alamat dari node-node yang dibuat tadi!

Dengan menggunakan Perintah berikut, node baru akan ditambahkan:

```
insertAfter(head->next->next,5);
    push(&head,9);
    append(&head,2);
    insertAfter(head->next->next,3);

    printf("\nModified DLL is: ");
    printList(head);
```

Hasil percobaan:

```
Praktikum > PS 1.2.cpp > main()
32  void append(struct Node** head_ref, int new_data) {
44          last->next=new_node;
45          new_node->prev=last;
46          return;
47  }
48  void printList(struct Node* node) {
49      struct Node* last;
50      printf("\nTraversal in forward direction \n");
51      while(node !=NULL){
52          printf(" %d", node->data);
53          last=node;
54          node=node->next;
55      }
56      printf("\nTraversal in reverse direction \n");
57      while(last !=NULL){
58          printf(" %d", last->data);
59          last=last->prev;
60      }
61  }
62  int main() {
63      struct Node* head=NULL;
64      append(&head, 6);
65      push(&head,7);
66      push(&head,1);
67      append(&head,4);
68      insertAfter(head->next,8);
69      printf("Created DLL is: ");
70      printList(head);
71
72      insertAfter(head->next->next,5);
73      push(&head,9);
74      append(&head,2);
75      insertAfter(head->next->next,3);
76
77      printf("\nModified DLL is: ");
78      printList(head);
79      getchar();
80      return 0;
81  }
82

PROBLEMS   OUTPUT   PORTS   DEBUG CONSOLE
Loaded 'C:\WINDOWS\System32\ucrtbase.dll'. Symbols loaded.
>
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  &
'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cp
ptools-1.19.9-win32-x64\debugAdapters\bin\WindowsD
ebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-r
mhbupz.4iv' '--stdout=Microsoft-MIEngine-Out-ujuu
axuf.2ks' '--stderr=Microsoft-MIEngine-Error-2jdos
olw.emb' '--pid=Microsoft-MIEngine-Pid-mhtgzp3.up
b' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--inte
rpreter=mi'
Created DLL is:
Traversal in forward direction
 1 7 8 6 4
Traversal in reverse direction
 4 6 8 7 1
Modified DLL is:
Traversal in forward direction
 9 1 7 3 8 5 6 4 2
Traversal in reverse direction
 2 4 6 5 8 3 7 1 9
```

Output program:

```
Created DLL is:
Traversal in forward direction
 1 7 8 6 4
Traversal in reverse direction
 4 6 8 7 1
Modified DLL is:
Traversal in forward direction
 9 1 7 3 8 5 6 4 2
Traversal in reverse direction
 2 4 6 5 8 3 7 1 9
```

2. **Percobaan 2 – Menghapus Sebuah Node Pada Doubly Linked List**

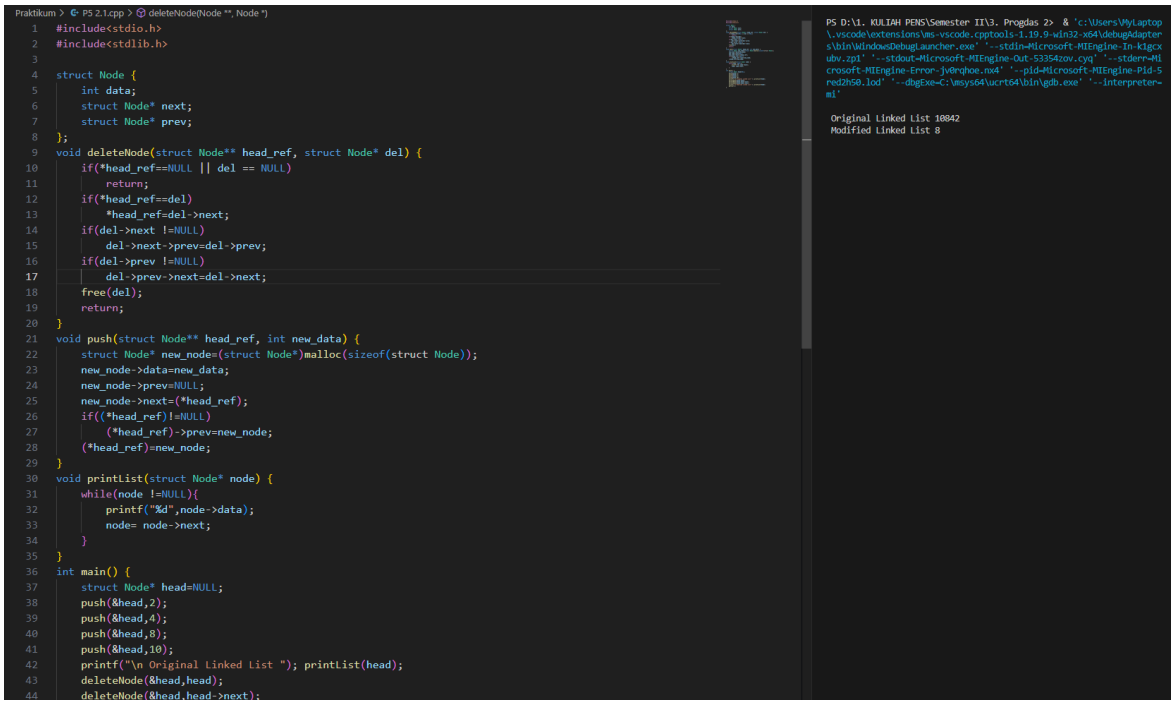
Pada percobaan ini, anda akan mencoba menghapus node tertentu dari sebuah DLL.

```
#include <stdio.h>
#include <stdlib.h>

/* a node of the doubly linked list */
struct Node {
	int data;
	struct Node* next;
	struct Node* prev;
};

/* Function to delete a node in a Doubly Linked List.
head_ref --> pointer to head node pointer.
del --> pointer to node to be deleted. */
void deleteNode(struct Node** head_ref, struct Node* del)
{
	/* base case */
	if (*head_ref == NULL || del == NULL)
		return;

	/* If node to be deleted is head node */
	if (*head_ref == del)
		*head_ref = del->next;

	/* Change next only if node to be deleted is NOT the last node */
	if (del->next != NULL)
		del->next->prev = del->prev;

	/* Change prev only if node to be deleted is NOT the first node */
	if (del->prev != NULL)
		del->prev->next = del->next;

	/* Finally, free the memory occupied by del*/
	free(del);
	return;
}

/* UTILITY FUNCTIONS */
/* Function to insert a node at the beginning of the Doubly Linked List */
```

```
void push(struct Node** head_ref, int new_data)
{
	/* allocate node */
	struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

	/* put in the data */
	new_node->data = new_data;

	/* since we are adding at the beginning,
	prev is always NULL */
	new_node->prev = NULL;

	/* link the old list off the new node */
	new_node->next = (*head_ref);

	/* change prev of head node to new node */
	if ((*head_ref) != NULL)
		(*head_ref)->prev = new_node;

	/* move the head to point to the new node */
	(*head_ref) = new_node;
}

/* Function to print nodes in a given doubly linked list
This function is same as printList() of singly linked list */
void printList(struct Node* node)
{
	while (node != NULL) {
		printf("%d ", node->data);
		node = node->next;
	}
}

/* Driver program to test above functions*/
int main()
{
	/* Start with the empty list */
	struct Node* head = NULL;

	/* Let us create the doubly linked list 10<->8<->4<->2 */
	push(&head, 2);
```

```
        push(&head, 4);
        push(&head, 8);
        push(&head, 10);

        printf("\n Original Linked list ");
        printList(head);

        /* delete nodes from the doubly linked list */
        deleteNode(&head, head); /*delete first node*/
        deleteNode(&head, head->next); /*delete middle node*/
        deleteNode(&head, head->next); /*delete last node*/

        /* Modified linked list will be NULL<-8->NULL */
        printf("\n Modified Linked list ");
        printList(head);

        getchar();
}
```

Hasil percobaan:




Output program:

```
Original Linked List 10842
Modified Linked List 8
```

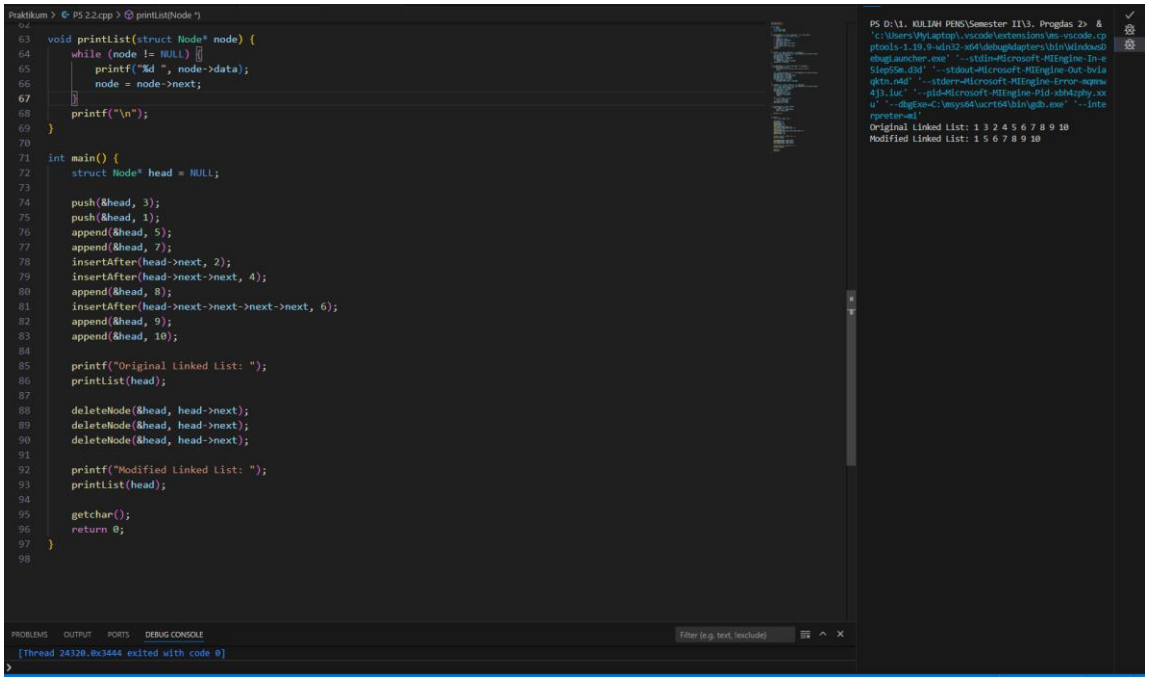
Dari percobaan di atas, cobalah untuk menambahkan 10 node baru terlebih dahulu (lokasi boleh acak, bisa di depan, di tengah ataupun di belakang), kemudian lakukan penghapusan terhadap 3 data yang baru anda masukkan secara acak!

(Untuk melakukan langkah ini, gabungkan fungsi insertAfter() dan append() pada percobaan 1 di atas ke dalam program anda sekarang.)

Dengan menggunakan Perintah berikut, 10 node baru akan ditambahkan dan dihapus 3:

```
push(&head, 3);
    push(&head, 1);
    append(&head, 5);
    append(&head, 7);
    insertAfter(head->next, 2);
    insertAfter(head->next->next, 4);
    append(&head, 8);
    insertAfter(head->next->next->next->next, 6);
    append(&head, 9);
    append(&head, 10);
    printf("Original Linked List: ");
    printList(head);
    deleteNode(&head, head->next);
    deleteNode(&head, head->next);
    deleteNode(&head, head->next);
    printf("Modified Linked List: ");
    printList(head);
```

Hasil percobaan:




Output program:

```
Original Linked List: 1 3 2 4 5 6 7 8 9 10
Modified Linked List: 1 5 6 7 8 9 10
```

3. **Percobaan 3 – Membalik Urutan Node Pada Doubly Linked List**

   Pada percobaan ini, seluruh node dalam DLL dapat dibalik secara iterative dengan menggunakan fungsi reverse().

```
/* Program to reverse a doubly linked list */
#include <stdio.h>
#include <stdlib.h>

/* a node of the doubly linked list */
struct Node
{
int data;
struct Node *next;
struct Node *prev;
};
```

```
/* Function to reverse a Doubly Linked List */
void reverse(struct Node **head_ref)
{
	struct Node *temp = NULL;
	struct Node *current = *head_ref;

	/* swap next and prev for all nodes of
	doubly linked list */
	while (current != NULL)
	{
	temp = current->prev;
	current->prev = current->next;
	current->next = temp;
	current = current->prev;
	}

	/* Before changing head, check for the cases like empty
		list and list with only one node */
	if(temp != NULL )
		*head_ref = temp->prev;
}

/* UTILITY FUNCTIONS */
/* Function to insert a node at the beginging of the Doubly Linked List */
void push(struct Node** head_ref, int new_data)
{
	/* allocate node */
	struct Node* new_node =
				(struct Node*) malloc(sizeof(struct Node));

	/* put in the data */
	new_node->data = new_data;

	/* since we are adding at the beginning,
	prev is always NULL */
	new_node->prev = NULL;

	/* link the old list off the new node */
	new_node->next = (*head_ref);

	/* change prev of head node to new node */
```

```
        if((*head_ref) != NULL)
        (*head_ref)->prev = new_node ;

        /* move the head to point to the new node */
        (*head_ref) = new_node;
}

/* Function to print nodes in a given doubly linked list
This function is same as printList() of singly linked list */
void printList(struct Node *node)
{
while(node!=NULL)
{
printf("%d ", node->data);
node = node->next;
}
}

/* Driver program to test above functions*/
int main()
{
/* Start with the empty list */
struct Node* head = NULL;

/* Let us create a sorted linked list to test the functions
Created linked list will be 10->8->4->2 */
push(&head, 2);
push(&head, 4);
push(&head, 8);
push(&head, 10);

printf("\n Original Linked list ");
printList(head);

/* Reverse doubly linked list */
reverse(&head);

printf("\n Reversed Linked list ");
printList(head);

getchar();
```

```
}
```

Hasil percobaan:

Praktikum > P5 3.1.cpp > push(Node **, int)

```c
#include<stdio.h>
#include<stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *prev;
};
void reverse(struct Node **head_ref){
    struct Node *temp=NULL;
    struct Node *current=*head_ref;
    while(current !=NULL) {
        temp=current->prev;
        current->prev=current->next;
        current->next=temp;
        current=current->prev;
    }
    if(temp !=NULL)
        *head_ref=temp->prev;
}
void push(struct Node** head_ref,int new_data) {
    struct Node* new_node= (struct Node*)malloc(sizeof(struct Node));
    new_node->data=new_data;
    new_node->prev=NULL;
    new_node->next=(*head_ref);
    if((*head_ref) !=NULL)
    (*head_ref)->prev=new_node;
    (*head_ref)=new_node;
}
void printList(struct Node *node) {
    while(node!=NULL) {
        printf("%d ",node->data);
        node=node->next;
    }
}
int main() {
    struct Node* head=NULL;
    push(&head,2);
    push(&head,4);
    push(&head,8);
    push(&head,10);
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\Users\MyLaptop
\.vscode\extensions\ms-vscode.cpptools-1.19.9-win32-x64\debugAdapter
s\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-r3cu1
hzo.13g' '--stdout=Microsoft-MIEngine-Out-kpizdjSh.fvy' '--stderr=Mi
crosoft-MIEngine-Error-p4svtrpi.psz' '--pid=Microsoft-MIEngine-Pid-l
tlcyc0a.qbg' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=
mi'

 Original Linked List 10 8 4 2
 Reversed Linked List 2 4 8 10
```

Output program:

```
Original Linked List 10 8 4 2
Reversed Linked List 2 4 8 10
```

## 4. Percobaan 4 – Mengurutkan/Sorting Doubly Linked List

Pada percobaan ini, data-data dalam node-node dari sebuah DLL dapat diurutkan menggunakan Teknik Quicksort.

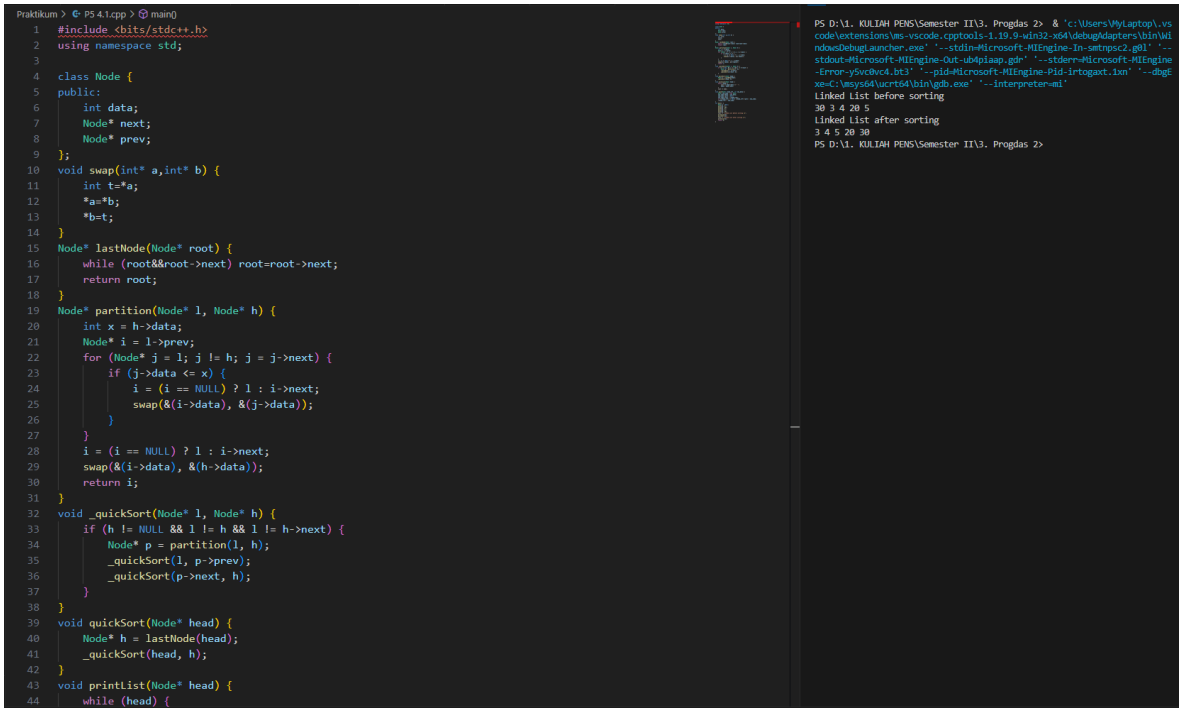
```
// A C++ program to sort a linked list using Quicksort
#include <bits/stdc++.h>
using namespace std;

/* a node of the doubly linked list */
class Node
{
	public:
	int data;
	Node *next;
	Node *prev;
};

/* A utility function to swap two elements */
void swap ( int* a, int* b )
{ int t = *a; *a = *b; *b = t; }

// A utility function to find
// last node of linked list
Node *lastNode(Node *root)
{
	while (root && root->next)
		root = root->next;
	return root;
}

/* Considers last element as pivot, places the pivot element at its correct position in
sorted array, and places all smaller (smaller than pivot) to left of pivot and all greater
elements to right of pivot */
Node* partition(Node *l, Node *h)
{
	// set pivot as h element
	int x = h->data;

	// similar to i = l-1 for array implementation
```

```
	Node *i = l->prev;

	// Similar to "for (int j = l; j <= h- 1; j++)"
	for (Node *j = l; j != h; j = j->next)
	{
		if (j->data <= x)
		{
			// Similar to i++ for array
			i = (i == NULL)? l : i->next;

			swap(&(i->data), &(j->data));
		}
	}
	i = (i == NULL)? l : i->next; // Similar to i++
	swap(&(i->data), &(h->data));
	return i;
}

/* A recursive implementation of quicksort for linked list */
void _quickSort(Node* l, Node *h)
{
	if (h != NULL && l != h && l != h->next)
	{
		Node *p = partition(l, h);
		_quickSort(l, p->prev);
		_quickSort(p->next, h);
	}
}

// The main function to sort a linked list.
// It mainly calls _quickSort()
void quickSort(Node *head)
{
	// Find last node
	Node *h = lastNode(head);

	// Call the recursive QuickSort
	_quickSort(head, h);
}

// A utility function to print contents of arr
```

```
void printList(Node *head)
{
	while (head)
	{
		cout << head->data << " ";
		head = head->next;
	}
	cout << endl;
}

/* Function to insert a node at the beginning of the Doubly Linked List */
void push(Node** head_ref, int new_data)
{
	Node* new_node = new Node; /* allocate node */
	new_node->data = new_data;

	/* since we are adding at the
	beginning, prev is always NULL */
	new_node->prev = NULL;

	/* link the old list off the new node */
	new_node->next = (*head_ref);

	/* change prev of head node to new node */
	if ((*head_ref) != NULL) (*head_ref)->prev = new_node ;

	/* move the head to point to the new node */
	(*head_ref) = new_node;
}

/* Driver code */
int main()
{
	Node *a = NULL;
	push(&a, 5);
	push(&a, 20);
	push(&a, 4);
	push(&a, 3);
	push(&a, 30);

	cout << "Linked List before sorting \n";
```

```
        printList(a);

        quickSort(a);

        cout << "Linked List after sorting \n";
        printList(a);

        return 0;
}
```

Hasil percobaan:




Output program:

```
Linked List before sorting
30 3 4 20 5
Linked List after sorting
3 4 5 20 30
```

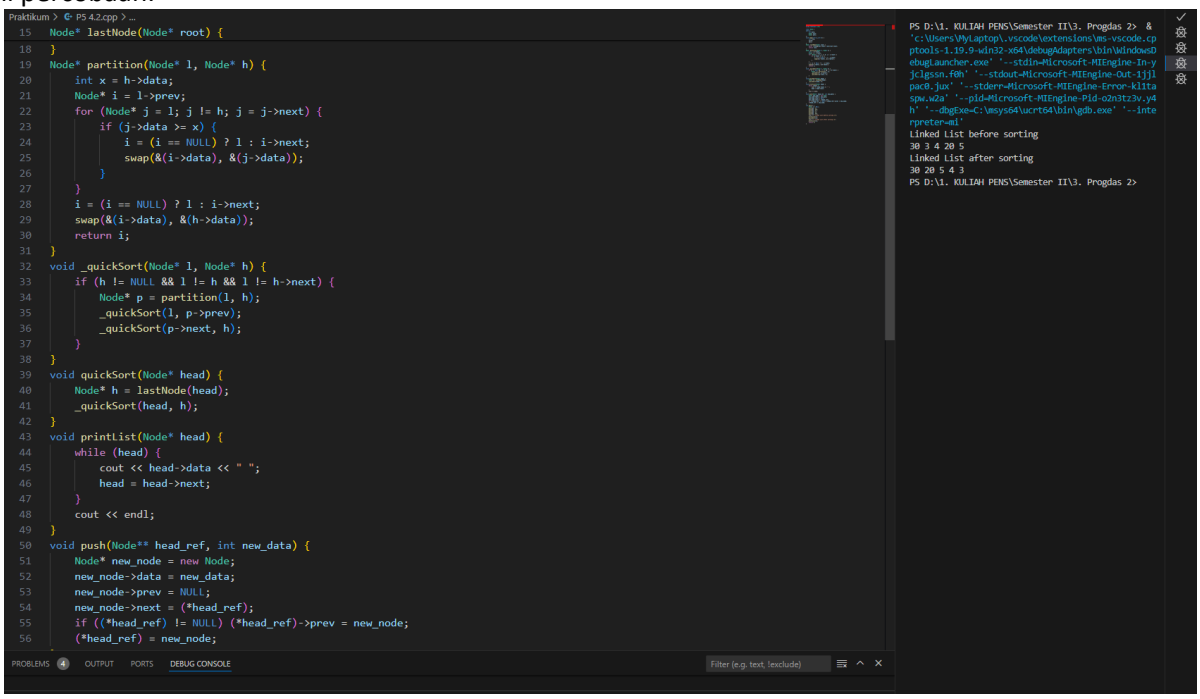
Percobaan di atas adalah program untuk mengurutkan data dari node DLL secara ascendant. Modifikasilah program tersebut sehingga dapat mengurutkan data dari node DLL secara descendent!

Dengan mengubah function partition, sebelumnya menggunakan operator <= menjadi >=.

```
Node* partition(Node* l, Node* h) {
    int x = h->data;
    Node* i = l->prev;
    for (Node* j = l; j != h; j = j->next) {
        if (j->data >= x) {
            i = (i == NULL) ? l : i->next;
            swap(&(i->data), &(j->data));
```

```
        }
    }
    i = (i == NULL) ? l : i->next;
    swap(&(i->data), &(h->data));
    return i;
}
```

Hasil percobaan:




Output percobaan:

```
Linked List before sorting
30 3 4 20 5
Linked List after sorting
30 20 5 4 3
```

Terlihat perbedaannya dari output dengan program yang menggunakan ascendant

```
Linked List before sorting
30 3 4 20 5
Linked List after sorting
3 4 5 20 30
```